

Fanela Central System

From Browser Prototype to Production Platform
Project Briefing for Meeting

Project Team

27 September 2026

Source documents:

`docs/superpowers/specs/2026-09-27-fanela-central-system-design.md` (design spec)

`Fanela_Internal_Production_Planner_v11.html` (prototype)

`Fanela_Planner_v11_Operations_Guide.pdf` (operations guide)

Contents

1	Executive Summary	3
2	What We Have Right Now	3
3	What Stays: The Business Rules	4
3.1	Customer master and order snapshots	4
3.2	Job structure and product grid	5
3.3	Commercial visibility (money is compartmentalised)	5
3.4	Artwork approval	5
3.5	Embroidery swatching – the most complex workflow	5
3.6	Department stages and job completion	6
3.7	Readiness traffic lights (computed, never typed in)	6
3.8	Dispatch and shipments	7
3.9	DPD – honest manual mode first	7
3.10	Excel – useful, but never the truth	7
3.11	Audit and stock history – two separate append-only logs	8
4	What Changes: The Plumbing	8
5	How the New System Is Built	8
5.1	Deployment on the VPS	9
5.2	Inside the application: modules	9
5.3	The five checks every API request passes	9
5.4	Technology choices and why (plain words)	10
6	Who Can Do What	10
7	How Data Is Protected	11
8	Moving Across: Migration and Testing	13
9	Delivery Phases	14
10	Decisions Taken and Open Questions	15

1 Executive Summary

In plain words

Today we run the production planner as a single HTML file opened in one browser. All data, orders, passwords and history sit inside that browser. If the browser cache is cleared, everything disappears. Two people cannot work on the same job at the same time, and nothing stops a user from changing something they should not.

We will keep every business rule the planner already teaches us – exactly as they are – and move them into a proper central web system: one database on our own server, real logins, real permissions enforced by the server, automatic backups, and room to connect DPD and Xero later. The prototype is the specification; the plumbing is what changes.

The project in one line

Preserve the operational logic of v11. Replace the browser-local trust model with a secure, central, multi-user system.

Headline decisions already taken

Topic	Decision
Architecture	Single application (modular monolith). One code base, clear internal modules. No microservices.
Stack	Next.js 15 (TypeScript) + PostgreSQL 16, Drizzle ORM
Authentication	Built into the system: email + password (Argon2id), one-time-code (TOTP) multi-factor for key roles
File storage	MinIO (private S3-compatible storage, runs on our server)
Background jobs	pg-boss (job queue inside PostgreSQL – no Redis needed)
Hosting	Our own VPS with Docker Compose: Caddy (HTTPS) + app + worker + PostgreSQL + MinIO
Scope of first release	Full operational MVP (customers, jobs, stock, stages, artwork, swatching, dispatch, audit, Excel)
After go-live	DPD API integration, then Xero integration
Migration	One clean JSON backup from the prototype, imported with reconciliation

What this document covers

Sections 2 to 10 explain, first in everyday language and then in full detail: what we have now, what stays, what changes, how the new system is built, who can do what, how data is protected, how we move across, and what we still need to decide together.

2 What We Have Right Now

In plain words

The Fanela Planner v11 is one HTML file (about 300 KB) that runs inside a web browser. It has been improved at least eleven times, and each version added real factory rules: how orders are recorded, how sizes are allocated, how artwork is approved, how embroidery samples block production, how shipments are tracked.

It works – but only inside the browser that opened it. Data lives in the browser's local storage. Accounts and passwords are stored locally. Attachments are limited to 2 MB. There is no sync between computers, no central backup, and no server to stop a user from bending the rules. Clearing browser storage deletes everything.

What the prototype does well

- Captures years of hard-won operational detail that would take months of workshops to rediscover.
- Encodes complex controls: customer snapshots, artwork approval, embroidery swatch gating, department queues, dispatch records, append-only stock history.
- Honest about its limits – for example it states clearly that DPD booking is manual and there is no live API.

What limits it

Limitation	Consequence
Browser-local storage	One machine, one browser. No teamwork, no sync
Local accounts and passwords	No real identity, no multi-factor, no central revocation
Client-side permissions	Anyone with developer tools can bypass role restrictions
Manual backups	Easy to lose everything; no point-in-time recovery
Attachments in browser (2 MB)	Photos and labels barely fit; lost with the browser
No concurrency handling	Two users editing the same job silently overwrite each other
Audit stored locally	Not tamper-resistant; can vanish with the browser
DPD and Xero	Manual, disconnected processes

Careful

Hosting the existing HTML file on a web server would solve none of the above. It would still be browser-local storage with no multi-user safety. That path is explicitly ruled out (see anti-patterns, Section 8).

3 What Stays: The Business Rules

In plain words

Everything in this section comes straight from the v11 prototype and is treated as a non-negotiable requirement. The new system must behave the same way – same gates, same approvals, same queues – so that staff recognise it immediately. If a rule is ever changed, it is a deliberate business decision, never an accident of the rebuild.

3.1 Customer master and order snapshots

Rule	Why it matters
Customer master stores name, contact, email, phone, billing address, default dispatch address, default dispatch method, account reference and notes	No re-typing for repeat customers
Selecting a customer copies their defaults onto the job as a snapshot	Every order is frozen as it was taken
Job contact/address/method are snapshots, not live links	Old orders never change when the customer moves house
“Deliberately update customer master” is a separate, confirmed, audited action	Routine order editing cannot corrupt master data

Database consequence: customers table is separate from job_contact_snapshot and job_dispatch_snapshot. The update-master action is its own API call that always writes an audit entry.

3.2 Job structure and product grid

- Job fields: Customer PO, Print/Job Name, Order Date, Order Type (Bulk, POD, Repeat, Sample), Priority (1 = highest), assigned staff, process date, dispatch date/time.
- Size grid XS–5XL per product/colour line, stored as a structured table.
- SKU = Manufacturer/Master Product Code; Supplier SKU kept as a separate field.
- **Outstanding quantity = ordered – received**, computed from stock events, negative values allowed (over-delivery is real and must show).
- Search by job number, customer, PO, print name, master SKU, supplier SKU.
- Queue order: dispatch deadline, then priority, then process date.

3.3 Commercial visibility (money is compartmentalised)

Field	Enter	View	Never sees
Buying costs	Admin, Operations	Admin, Operations; Director in reports	Office, Dispatch, Packing, Department Operator
Selling price / tax	authorised commercial roles only	same set	everyone else

These are **field-level server-side permissions**: the API removes restricted fields from every response. Hiding columns in the interface is not enough.

3.4 Artwork approval

- Per job: proof reference, version, approval status, approved by/date, Pantone notes, uploads, links, positions.
- Only Admin and Operations approve or revise approved artwork. Everyone else is read-only.
- **Revising approved artwork resets it to Draft** and writes an audit entry with before/after – the revised work must be approved again.
- Operators see production references and approval state, but cannot change them.

State machine:

Draft → Awaiting Approval → Approved | Rejected Approved $\xrightarrow{\text{revision}}$ Draft (new version, audited)

3.5 Embroidery swatching – the most complex workflow

In plain words

Embroidery jobs need an approved sample (swatch) before full production starts. The embroiderers make a sample, it gets approved or rejected with a reason, rejected samples stay visible for learning, and approved samples can never be edited. If a required sample is not yet approved, the server refuses to let the embroidery stage start, progress or finish. Operations can waive the requirement, but only with a recorded reason.

- Swatch required by default for embroidery jobs; waiver by Admin/Ops with audit.
- Each attempt records: sample quantity, embroidery file reference, thread colours, stitch count, placement, machine, notes, photos.
- Workflow: start (records operator and time) → complete / request approval (records user and time) → approve / reject (reason required) / request re-swatch (reason required).

- Rejected attempts stay visible with reason, photos and timestamps.
- Approved attempts are **immutable**; another sample means a new attempt.
- **Production gate**: if a swatch is required and the latest attempt is not approved, every embroidery stage transition (start, progress, complete) is rejected by the server with a state error.
- Queue states: waiting, swatching, awaiting approval, ready for production, in production.

3.6 Department stages and job completion

- Each job has one stage per required department; stages progress independently.
- Department users see only their own stages; the job detail page shows all statuses for coordination.
- An operator may update **only their own department's stage** (Admin and Operations can override).
- Stage data: notes, waste quantity, reprint quantity, quantity, progress, remaining pieces, start and finish times, completing user.
- Completing one stage removes it from that department's Active queue; other stages stay open.
- Total prints = pieces × selected print positions.
- **A job only completes when every required stage is complete AND dispatch is complete.** Dispatch can never be skipped.

3.7 Readiness traffic lights (computed, never typed in)

In plain words

Every job shows a colour. Green means everything needed to start work is ready: stock received, screens made, and (for embroidery) the sample approved. White means nothing that the job actually needs is ready yet. Amber is in between. The colour is calculated by the server every time something changes – nobody can set it by hand.

The calculation counts only the gates that apply to the job:

```
active gates = stock? screens? swatch?
green       = all active gates pass
white      = no active gate passes (or job has nothing required yet)
amber      = some pass, some don't
```

- Stock gate: every line received $\geq$ ordered.
- Screens gate: made $\geq$ required. A **blank** required field means “not yet specified” and therefore *not ready* – blank is never treated as zero.
- Swatch gate: not required, or latest attempt approved.

Developer note: empty-job trap

The zero-gate case must be checked *before* any gate evaluation: `if (totalRequiredGates === 0) return 'White'`. JavaScript array methods such as `[] .every(...)` return `true` on empty arrays – relying on them makes a job with no products, screens or swatches turn **Green** the moment it is created.

3.8 Dispatch and shipments

Rule	Detail
Four methods with method-specific fields	Collection (contact, phone, date, time, confirmation); DPD (consignment, parcels, weights, service, instructions, labels); Same Day (courier, driver, time, ETA, reference); Fanela Van (driver, route, departure, ETA)
Who can edit	Dispatch + Admin/Operations; Office sets planned method/address at entry; Director and Packing read-only
Booking creates a shipment record	“Record booking/collection arrangement” saves a shipment; “record final dispatch” records departure, user, time
Guard rails	Other required stages must be finished first; Collection needs confirmation; the Dispatch stage completes only via an explicit “ Finalise dispatch / close shipments ” action – one dispatched shipment among several must never auto-close the stage, and finalise is rejected while any shipment is still open (unless explicitly abandoned, with a reason, audited)
Part dispatch	While a job is open, one shipment can be dispatched and others follow; multiple shipments are kept
Cancellation = event, never deletion	Void records a reason against the shipment; the booking stays; no carrier-side cancellation unless later integrated

Shipment states:

Draft → Booking Arranged → Booked → Labels Attached → Print Requested →
Labels Printed → Dispatched / Collected

Any non-final state can move to Cancelled/Void with a reason.

3.9 DPD – honest manual mode first

No live DPD booking in the first release. The manual workflow from the prototype is kept, but server-stored: book with DPD externally, enter consignment and parcel details, save the booking *first*, attach returned label files (default limit 25 MB, configurable), log print requests without ever claiming physical printing succeeded, then confirm “labels printed” manually. Booking, tracking and labels survive failures and can be retried. A future API mode adds real bookings; **manual mode remains as fallback**.

3.10 Excel – useful, but never the truth

- Exports (all jobs, filtered, department, stock/shortage, swatches, shipments, audit) are generated on the server and filtered by the caller’s permissions.
- Historical quantities in reports are snapshots – **never summed as stock movements**.
- Imports are add-only: checked before confirmation, duplicates skipped, imported artwork starts as Draft.
- Never reconstructed from Excel: historical swatch approvals, shipment records, photos, audit history.

3.11 Audit and stock history – two separate append-only logs

1. **Stock history** – receipts, adjustments, corrections with reason, user, timestamp. Never edited, never deleted.
2. **Operational audit** – job/header, priority, order lines, artwork, screens, swatch, stage, customer master, dispatch, shipment and approval changes, with before/after snapshots.

Department users see the entries relevant to their work; sensitive commercial changes stay restricted.

4 What Changes: The Plumbing

In plain words

Nothing in Section 3 is an accident – it is business behaviour. Everything in this section is a *technical* limitation of running inside a browser. These are the parts we replace.

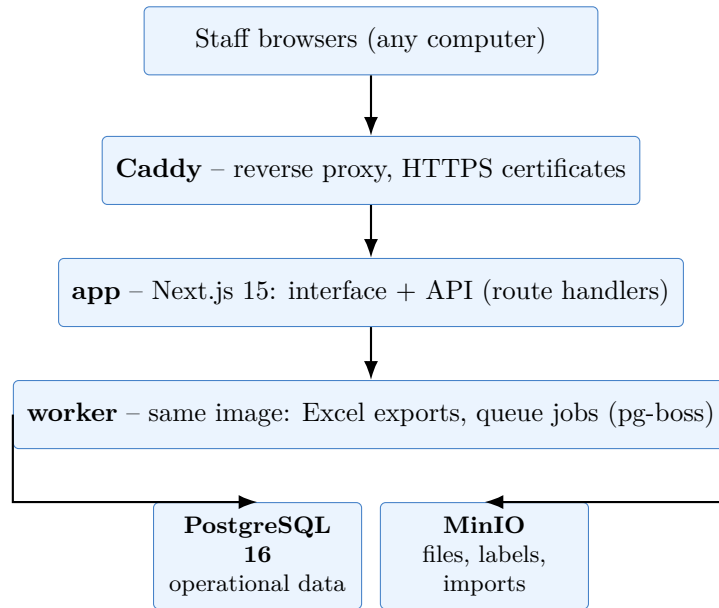
Today (v11)	New system	Priority
All data in browser local storage	Central PostgreSQL database	Critical
No sync between computers	Central web app, everyone works simultaneously	Critical
Local accounts with local password hashes	Server authentication, Argon2id + TOTP multi-factor	Critical
Permissions only hide buttons	Server-side RBAC at every API endpoint, including field-level rules	Critical
Clearing the browser deletes everything	Nightly backups copied off the server	Critical
Attachments in browser, 2 MB limit	MinIO object storage, 25 MB default, access-checked links	High
No concurrent-edit handling	Version numbers; conflicting saves rejected with a clear message (HTTP 409)	High
Audit stored locally	Server append-only audit with before/after snapshots	High
Migration depends on browser access	Controlled import from the JSON backup with reconciliation	High
DPD manual-only, button disabled	Manual mode kept (server-stored); API adapter later	Medium
No Xero	Outbox-pattern adapter after go-live	Medium
Browser print dialog	Stored label files, print-request log, confirmed-print events	Medium
Excel done in the browser	Server-generated exports, staged/validated imports	Medium
No monitoring	Structured logs, health checks, uptime monitoring	Medium
No notifications	Later: e-mail/in-app alerts for approvals and blocked stages	Low

5 How the New System Is Built

In plain words

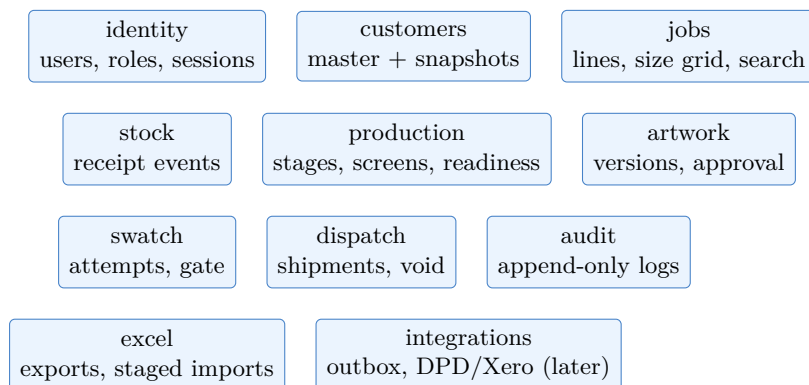
We build one application, not many. It runs on our own server inside Docker containers: a web server that handles HTTPS, the application itself, a background worker for slow jobs like Excel exports, PostgreSQL as the single source of truth, and MinIO for files. Every request is checked on the server: who are you, what are you allowed to do, is this change valid in the current state, has someone else already edited this record.

5.1 Deployment on the VPS



All five containers are defined in a single Docker Compose file, started together, and restart on failure. Nightly jobs copy a database dump and the file store off the server.

5.2 Inside the application: modules



- Route handlers stay thin: parse input → check permission → call the module → serialise the reply.
- All business rules live inside module services – the state machines are enforced in exactly one place.
- A static-analysis tool (dependency-cruiser) runs in CI and fails the build if a module reaches into another module's internals.

5.3 The five checks every API request passes

1. **Authenticated?** – valid server-side session.
2. **Role permits it?** – permission catalogue seeded in the database.
3. **Scope and state valid?** – department ownership, workflow transition allowed, field-level stripping applied.
4. **No concurrent edit?** – record version matches, otherwise HTTP 409 with the current data.
5. **Atomic** – entity change and its audit entry commit in one transaction.

5.4 Technology choices and why (plain words)

Choice	Reason
Next.js 15 (TypeScript)	One language, one code base for interface and API; huge ecosystem; well suited to server-side checks
PostgreSQL 16	The single source of truth; strong transactions; supports the job queue itself
Drizzle ORM	Type-safe SQL with migrations kept in the repository
Auth.js + Argon2id + TOTP	No third-party identity service; passwords hashed with a memory-hard algorithm; multi-factor for key roles
pg-boss (no Redis)	Background jobs (exports, later integrations) queue inside PostgreSQL – one less service on a small VPS
MinIO	S3-compatible file storage that runs on our own hardware; files stay private behind permission checks
Caddy	Automatic HTTPS certificates with almost no configuration
Modular monolith (no microservices)	One factory, one deployment: faster to build, easier to test, cheaper to run; modules keep the code clean and allow later extraction of integration adapters

Implementation notes for the engineering team

- **Auth.js multi-step MFA.** The Credentials provider's `authorize()` handles the password only – Auth.js has no native two-step MFA. Use a staging flow: Step 1 validates the password and issues a short-lived, scope-restricted `pending_mfa` token; Step 2 validates the TOTP against that token and exchanges it for the real session cookie. Users are never asked for a TOTP code before their password is verified, and a `pending_mfa` token cannot call application endpoints.
- **Database security migrations.** Drizzle manages tables, columns and indexes – not roles, `GRANT/REVOKE` or row-level security. Keep a separate folder of raw SQL migrations (append-only roles, `FORCE ROW LEVEL SECURITY` policies) executed after Drizzle migrations: schema migration → security migration → app deploy.

6 Who Can Do What

In plain words

Seven roles: Admin, Operations Manager, Office, Director, Dispatch, Packing and Department Operator. The interface hides buttons the user cannot use, but the *server* is the judge – every API call is checked independently, including which columns of data the reply may contain (for example, costs never leave the server for Office users).

Roles at a glance

Role	Scope
Admin	Everything: users, roles, backups, all jobs and data
Operations Manager	All jobs; approves artwork and swatches; edits dispatch; views costs; audit
Office	Creates/edits jobs, customer master, planned dispatch, Excel. No buying costs
Director	Read-only oversight; costs in reports only; no editing
Dispatch	Dispatch details, bookings, final dispatch, void. No costs
Packing	Read-only dispatch information and relevant stages
Department Operator	Updates own department's stage only ; no costs; no approvals

Permission matrix

Key: **Y** yes **Y*** only with an extra grant **R** read-only **L** limited scope **N** no

Capability	Adm	Ops	Off	Dir	Dis	Pck	Dept
Manage users / roles / backups	Y	N	N	N	N	N	N
View customer master	Y	Y	Y	Y	Y	Y	N
Edit customer master	Y	Y	Y	N	N	N	N
Deliberately update master	Y	Y	N	N	N	N	N
Create / edit jobs	Y	Y	Y	R	L	R	N
Enter buying costs	Y	Y	N	N	N	N	N
View buying costs	Y	Y	N	Y*	N	N	N
View selling price / tax	Y*	Y*	Y*	Y*	N	N	N
Approve / revise artwork	Y	Y	N	N	N	N	N
Swatch create / start / complete	Y	Y	N	N	N	N	Y*
Swatch approve / reject / waive	Y	Y	N	N	N	N	N
Update a stage	Y	Y	N	N	N	N	Y*
Dispatch edit / book / final / void	Y	Y	plan	R	Y	R	N
Excel import / export	Y	Y	Y*	L	N	N	N
View audit	Y	Y	L	L	L	L	rel
Stock receipt entry	Y	Y	N	N	N	N	Y*

Y* means an additional grant exists (“authorised commercial” flag for prices, department assignment for stages/swatches).

How it is enforced

- The matrix lives in the database (`role_permissions`) and the same catalogue drives both the interface and the API checks – they cannot drift apart.
- Every mutating endpoint is covered by automated “permission probe” tests: call it as each role, expect exactly the right allow/deny.
- Costs and prices are stripped in the serialiser before the response is built, including in Excel exports (the worker decides columns at write time: Director’s copy includes costs, Office’s copy does not).

7 How Data Is Protected

In plain words

In a factory, history matters: who approved what, when a shipment left, how much stock arrived. The new system therefore never truly deletes important records – it cancels, voids or corrects them with a reason, and writes it down. Two logs (stock history and operational audit) are append-only, even against a developer making a mistake.

Integrity rules

Rule	How it is enforced
No hard delete of jobs, shipments, swatch attempts, approvals, stock or audit records Stock history and audit are append-only	Application has no delete path; database role denies DELETE Insert/select-only database role; significant changes store before/after snapshots as JSON
Approved swatch attempts are immutable	Application rejects edits <i>and</i> a database row-level security policy (<code>FORCE ROW LEVEL SECURITY</code> , policy status <code><> 'Approved'</code>) makes approved rows unwritable at the engine level – even if code bypasses the application layer. A blanket <code>REVOKE UPDATE</code> is not used: it would also block legitimate status transitions
Revising approved artwork resets it to Draft	State machine + audit entry in the same transaction
Embroidery blocked while a required swatch is unapproved A job completes only when all stages and dispatch are complete Void is not delete	Stage service re-checks the gate on every transition Computed; cannot be set directly The shipment keeps its booking rows; the void is an event with a reason
Concurrent edits are safe	Version numbers <i>scoped to the sub-entity being edited</i> : stages, shipments, swatch attempts and order lines have their own <code>version</code> columns, so a stock receipt never conflicts with an unrelated dispatch-date edit; mismatch returns 409 plus the current record
Multi-step actions are atomic	Stage completion, swatch decisions, stock receipts, booking/final dispatch and import confirmation all commit as one transaction
Blank requirements block readiness	A blank “screens required” field means <i>pending specification</i> , never zero – the gate stays closed
Files are private	Access only through the owning record’s permission check, via short-lived signed links; approved swatch files are write-once; labels survive voids

Sessions and login

- Email + password, hashed with Argon2id; multi-factor (TOTP) mandatory for Admin, Operations and Dispatch; recovery codes shown once.
- Server-side session cookie: http-only, secure, same-site; **2-hour idle timeout** (an unattended factory terminal re-logs-in well before shift end) and **12-hour absolute cap** (fresh login at the end of a long shift regardless of activity); logout destroys the session row. Idle must be strictly shorter than the absolute cap – a 12 h / 12 h pairing would make the idle rule dead code.
- Login attempts are rate-limited per email and per address, with progressive lockout; failures are logged.
- Open question for the meeting: confirm the 2-hour idle value and whether shared terminals also want a quick PIN re-lock screen (Section 10).

Backups

Nightly database dump plus a copy of the file store, both transferred off the server, retained at least 30 days (period to confirm), with a documented restore procedure. The prototype's JSON backup remains available as an export format.

8 Moving Across: Migration and Testing

In plain words

We migrate from one clean JSON backup taken from the current browser. The import is rehearsed first in a staging copy, counts are compared table by table, real users then run a checklist of everyday scenarios, and only after sign-off does the new system go live. The old file and backups are kept. We never invent history: if the prototype has no approval record, the new system will not either.

Migration steps

#	Action	Pass criterion
1	Inventory: find every browser/backup, export Excel reports for comparison	Newest backup identified
2	Parse the backup, count all entities, list orphans and duplicates	Counts recorded
3	Map prototype schema (version 3) to the new schema; keep IDs as <code>legacy_id</code>	Mapping document complete
4	Clean master data: de-duplicate customers/products, fill missing supplier SKUs (as a fix list, no invention)	Issues reviewed
5	Dry-run import into staging	All counts reconcile (customers, products, jobs open/closed, lines, stages, stock, swatches, shipments, audit, files)
6	Cut-over: freeze local editing, take a final JSON backup	Backup stored
7	Import into production, verify live jobs	20 open jobs spot-checked + full UAT below
8	Create user accounts, assign roles, train staff	Sign-off
9	Retire the prototype to read-only reference; keep backups ≥ 90 days	—

User acceptance scenarios (also written as automated tests)

- Create customer → job → defaults copied → snapshot edited → deliberately update master → old jobs unchanged.
- Receive stock → screens confirmed → readiness moves White → Amber → Green.
- Readiness edge cases: stock-only job with no stock = White (not Amber); blank screens field = never Green.
- Approve artwork → revise → resets to Draft with audit → re-approve.
- Swatch: create → start → complete → approve → embroidery unblocks.
- Swatch: reject with reason → embroidery stays blocked → new attempt → approve → unblocks.
- Waive swatch requirement → audit entry → embroidery unblocks.

- Wrong department tries a stage update → denied; own department works; other stages unaffected; all stages + dispatch complete → job completed.
- Booking → final dispatch → second shipment (part dispatch) → void first with reason → booking preserved → job stays open until the explicit **Finalise dispatch** action, which is rejected while any shipment is still open.
- Excel export role-filtered (no cost columns for Office); import is add-only (duplicates skipped, artwork starts as Draft).
- Two users edit the *same* sub-entity: the second save is rejected with a conflict message and the current data. Two users edit *different* sub-entities of one job (stock receipt vs. dispatch date): **both succeed** – no spurious conflicts.
- Permission probes: every mutating endpoint called as each role returns exactly the expected allow/deny.

What must NOT happen

Careful

1. Hosting the existing HTML file on a web server.
2. Letting Excel become the source of truth.
3. Rebuilding the operational logic from scratch instead of using v11.
4. Relying on client-side-only permissions.
5. Deleting bookings, attempts or audit records.
6. Starting with microservices.
7. Integrating DPD or Xero before the operational core is stable.
8. Inventing historical approvals during migration.

9 Delivery Phases

In plain words

We build in five phases. Nothing big-bang: each phase ends with something demonstrable. The system becomes safe to pilot after Phase 2, complete enough to go live after Phase 3, and connected to DPD and Xero after Phase 4. Week ranges are planning estimates from the assessment document and assume the team sizes assumed there.

Phase	Deliverables	Estimate
0 – Discovery & domain freeze	Formal rule register from the v11 assessment, entity-relationship model, state-machine diagrams, permission matrix confirmed with the client, data-quality report from the backup, migration mapping, DPD/Xero scope agreement	2–3 weeks
1 – Secure foundation	Central authentication with roles, user management, database, customer master, job creation, product grid with size allocation, search/sort, audit skeleton, file storage, backups, deployment and monitoring	4–6 weeks
2 – Operational core	Department stages with Active/Upcoming/Blocked/Completed views, stock receipts and outstanding quantities, screens readiness, print positions, traffic-light readiness, queue sorting, role-restricted stage updates, stage notes/waste/reprints, role-filtered Excel export, concurrency handling	6–8 weeks
3 – Approvals & control (go-live MVP)	Artwork approval with versions, swatch workflow with production blocking, dispatch and shipment records with part dispatch and void, central file storage, audit viewer, staged Excel import, export tooling, migration executed, training	4–6 weeks
4 – Integrations & optimisation	DPD API adapter (booking, labels, reprint, manual fallback), print request/confirmation workflow, Xero adapter with outbox pattern, dashboards, notifications, performance work	4–8 weeks

Cumulative rough timeline: limited pilot possible after ≈ 11 – 17 weeks; full go-live after ≈ 16 – 23 weeks; integrations complete by ≈ 20 – 31 weeks.

10 Decisions Taken and Open Questions

In plain words

This is the meeting page. The left column lists what has already been decided and needs no discussion. The right column lists what we still need from the people in this room.

Decisions already taken

Topic	Decision
Source of truth for behaviour	The v11 prototype: every operational rule is preserved
Architecture	Single modular monolith (Approach A), no microservices
Stack	Next.js 15 + TypeScript, PostgreSQL 16, Drizzle ORM
Authentication	Built-in (Auth.js + Argon2id + TOTP), no external identity provider
Background jobs	pg-boss inside PostgreSQL; no Redis
Files	MinIO on our server, private buckets, 25 MB default
Hosting	Self-hosted VPS, Docker Compose (Caddy, app, worker, PostgreSQL, MinIO)
First release scope	Full operational MVP as in Phase 3
DPD	Manual server-stored mode first; API adapter later, manual mode kept as fallback
Xero	After go-live, outbox pattern, never the operational database
Excel	Projection only: server exports, staged add-only imports
Deletion policy	No hard deletes of operational records; void/cancel/correct instead
Migration source	One clean JSON backup with full reconciliation
Design spec	docs/superpowers/specs/
2026-09-27-fanela-central-system-design.md (reviewed and patched)	
Implementation plan	Explicitly deferred (not part of this document)

Alternatives considered (for the record)

- **Buy off-the-shelf (PrintVis $\approx$ \$130–150/user/month, Odoo print add-ons)** – rejected: Fanela’s own rules would need paid customisation anyway, plus per-seat cost and lock-in; revisit only if the business wants a full ERP.
- **Microservices** – rejected: industry consensus puts the switch above ~ 30 engineers; modular monolith kept, extraction path stays open.
- **Laravel / Django instead of Next.js** – equally valid only if the implementing team is PHP/Python-native; TypeScript stack kept.
- **Serverless cloud hosting** – rejected: usage bills grow, data stays with a third party; one VPS is enough for one factory.

Open questions for this meeting

1. **Permission matrix sign-off** – especially: who exactly sees costs in reports (Director only?), who holds the “authorised commercial” flag for selling prices, and how far Office can set planned dispatch details.
2. **Final department list** – print, embroidery, warehouse, screens, dispatch, packing: is anything missing or misnamed?
3. **Readiness verification** – confirm the White/Amber/Green counting rules against how the prototype actually behaves (especially blank screens).
4. **Attachments** – is 25 MB per file right, and who may download which file types?
5. **Backups** – retention period (30 days proposed) and where the off-server copy should live.
6. **DPD and Xero timing** – confirm both are Phase 4, or pull one forward.
7. **Session behaviour on shared terminals** – proposed 2-hour idle timeout (12-hour absolute cap stays; an earlier 12/12 pairing made the idle rule dead code). Confirm 2 hours, or add a quick PIN re-lock screen as well?
8. **Data inventory** – confirm that only one machine holds prototype data and identify the definitive backup file to migrate from.